

Arquitecturas de software

Toma de decisiones arquitectónicas, tecnologías

Sandra Cano

Pontificia Universidad Católica de Valparaíso
OII436-1 Ingeniería Web Avanzada

Segundo semestre de 2026

A tener en cuenta

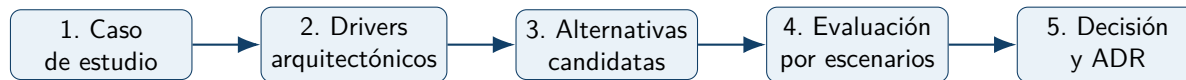
Mensaje central

No existe una arquitectura “mejor” en términos absolutos. Existe una arquitectura más adecuada para un contexto y un conjunto de atributos de calidad.

Idea clave

La arquitectura se justifica mediante evidencia, no por preferencia tecnológica.

Método para evaluar una arquitectura



Etapas	Pregunta guía
Drivers	¿Qué atributos son críticos: seguridad, escalabilidad, rendimiento, mantenibilidad, disponibilidad?
Alternativas	¿Qué estilos arquitectónicos podrían resolver el problema?
Escenarios	¿Cómo se comprobaría que la arquitectura responde al atributo?
Decisión	¿Qué se gana y qué se sacrifica?

Caso de Estudio: Sistema de Gestión de pedidos en Línea

Una empresa necesita una aplicación web para registrar usuarios, consultar productos y realizar pedidos. Inicialmente tendrá 2.000 usuarios, pero podría crecer. El equipo de desarrollo está formado por 5 personas y la empresa desea mantener bajos los costos de operación.

Caso de Estudio: Drivers Arquitectónicos

- Mantenibilidad;
- Escalabilidad;
- Seguridad;
- Disponibilidad;
- Complejidad operacional.

Drivers arquitectónicos seleccionados

Driver	¿A qué se refiere?
Mantenibilidad	Facilidad con la que el sistema puede ser modificado, corregido o ampliado sin producir efectos inesperados en otros componentes.
Escalabilidad	Capacidad del sistema para soportar un aumento de carga —usuarios, solicitudes, datos o procesamiento— manteniendo un nivel de servicio aceptable.
Seguridad	Capacidad para proteger datos, identidad y operaciones frente a accesos no autorizados, modificaciones indebidas o exposición de información.
Disponibilidad	Capacidad del sistema para continuar ofreciendo sus servicios cuando son requeridos, incluso ante fallos parciales o problemas en algunos componentes.
Complejidad operacional	Esfuerzo necesario para desplegar, configurar, monitorear, actualizar y recuperar el sistema en operación.

Driver arquitectónico: Disponibilidad

¿Qué es la disponibilidad?

Es la capacidad del sistema para estar **operativo y accesible** cuando los usuarios lo requieren, incluso cuando ocurren fallos parciales.

¿Qué aspectos considera?

- **Tolerancia a fallos:** evitar que la falla de un componente detenga todo el sistema.
- **Redundancia:** disponer de instancias o recursos alternativos.
- **Recuperación:** restaurar el servicio después de una falla.
- **Aislamiento de fallos:** limitar el impacto de un componente defectuoso.
- **Eliminación de puntos únicos de fallo:** evitar dependencias cuya caída interrumpa todo el sistema.
- **Continuidad del servicio:** mantener disponibles las funcionalidades críticas.

Pregunta orientadora

Si un componente falla, ¿qué parte del sistema continúa funcionando y cuánto tarda en recuperarse?

Driver arquitectónico: Complejidad operacional

¿Qué es la complejidad operacional?

Representa el **esfuerzo necesario para desplegar, configurar, supervisar, actualizar y recuperar** un sistema durante su operación.

¿Qué incluye?

- **Despliegue:** cantidad de aplicaciones o servicios que deben desplegarse.
- **Configuración:** variables, dependencias, redes, credenciales y entornos.
- **Monitoreo y observabilidad:** logs, métricas, trazas, alertas y health checks.
- **Actualizaciones:** coordinación de versiones y despliegues.
- **Gestión de fallos:** detección, diagnóstico y recuperación.
- **Infraestructura:** servidores, contenedores, balanceadores, colas, bases de datos y otros servicios.
- **Automatización:** CI/CD, aprovisionamiento y gestión de infraestructura.

Caso de Estudio: Evaluación x Escenarios

- Monolito modular.
- Microservicios.

Pregunta

¿Cuál arquitectura satisface mejor estos atributos. considerando las características concretas del caso?

Ventajas

- Menor complejidad inicial.
- Un único despliegue.
- Comunicación interna más simple.
- Facilita pruebas e integración.
- Menor infraestructura operacional.
- Adecuado para equipos pequeños.

Desventajas

- Escalamiento principalmente conjunto.
- Menor aislamiento ante fallos.
- Riesgo de aumentar el acoplamiento.
- El crecimiento puede dificultar los cambios si no se mantienen límites entre módulos.

Ventajas

- Escalamiento independiente.
- Despliegues independientes.
- Mayor aislamiento entre servicios.
- Permite evolucionar servicios de manera separada.
- Facilita equipos autónomos.

Desventajas

- Mayor complejidad operacional.
- Comunicación distribuida.
- Más difícil de probar y depurar.
- Requiere mayor observabilidad.
- Mayor infraestructura y automatización.

Método

Cada atributo de calidad recibe una **ponderación** según su importancia para el caso de estudio. Cada arquitectura se evalúa utilizando una escala de 1 a 5.

Valor	Interpretación
1	Muy desfavorable / no satisface el escenario
2	Desfavorable / lo satisface parcialmente
3	Aceptable
4	Favorable
5	Muy favorable / satisface ampliamente el escenario

Matriz de evaluación ponderada

Drivers identificados

A partir de los requisitos y restricciones del caso se seleccionaron cinco atributos relevantes para la decisión.

Driver arquitectónico	Ponderación
Mantenibilidad	25 %
Escalabilidad	20 %
Seguridad	20 %
Disponibilidad	15 %
Complejidad operacional	20 %
Total	100 %

Importante

La ponderación representa la **importancia relativa** de cada atributo para este caso de estudio.

Evaluación ponderada de las arquitecturas

Driver	Peso	Monolito modular		Microservicios	
		Valor	Pond.	Valor	Pond.
Mantenibilidad	25 %	5	1.25	4	1.00
Escalabilidad	20 %	3	0.60	5	1.00
Seguridad	20 %	4	0.80	4	0.80
Disponibilidad	15 %	3	0.45	5	0.75
Complejidad operacional	20 %	5	1.00	2	0.40
Resultado	100 %		4.10		3.95

$$P(A) = \sum_{i=1}^n w_i \times v_{i,A}$$

Resultado

Para las prioridades definidas en este caso, el **monolito modular** obtiene **4.10** y los **microservicios** **3.95**.

Justificación de las valoraciones

Driver	Monolito modular	Microservicios
Mantenibilidad	5. Módulos delimitados y menor complejidad para un equipo pequeño.	4. Buena separación de responsabilidades, pero aumenta la complejidad distribuida.
Escalabilidad	3. Puede escalar mediante réplicas, pero normalmente escala la aplicación completa.	5. Cada servicio puede escalar independientemente según su demanda.
Seguridad	4. Controles más centralizados y menor superficie de comunicación.	4. Permite controles independientes, pero aumenta la comunicación entre servicios.
Disponibilidad	3. Una falla grave puede afectar una mayor parte de la aplicación.	5. Un servicio puede fallar sin necesariamente detener todo el sistema, si existe tolerancia a fallos.
Complejidad operacional	5. Un despliegue, menor infraestructura y operación más sencilla.	2. Requiere múltiples despliegues, comunicación distribuida, monitoreo y automatización.

¿Qué significa el resultado?

Monolito modular

4.10

- Mayor mantenibilidad inicial.
- Menor complejidad operacional.
- Adecuado para un equipo pequeño.
- Escalabilidad más limitada.

Microservicios

3.95

- Mayor escalabilidad.
- Mayor disponibilidad potencial.
- Mayor independencia.
- Mayor complejidad operacional.

Conclusión

El monolito modular no es “mejor” que los microservicios.

Es la alternativa **más adecuada para los drivers, ponderaciones y restricciones de este caso.**

Arquitectura híbrida

Definición

Una arquitectura híbrida combina **dos o más estilos arquitectónicos** dentro de una misma solución, seleccionando cada enfoque según las necesidades y atributos de calidad del sistema.

Ejemplos de combinaciones

- Monolito modular + arquitectura orientada a eventos.
- Microservicios + comunicación basada en eventos.
- Monolito modular + componentes serverless.
- Microservicios con arquitectura hexagonal interna.
- REST para operaciones síncronas + eventos para procesamiento asíncrono.

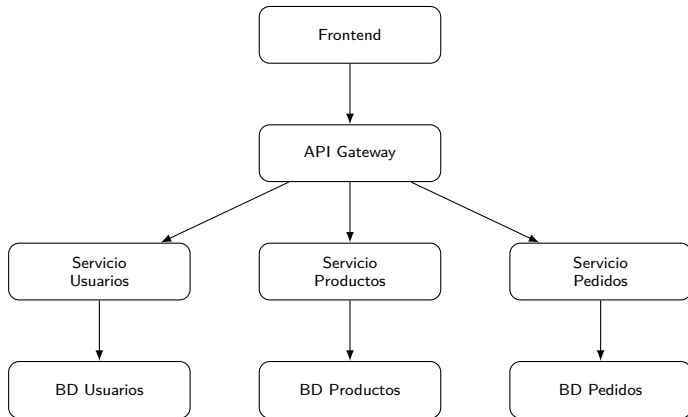
Idea clave

Una arquitectura híbrida no busca “mezclar tecnologías”, sino combinar decisiones arquitectónicas para responder mejor a diferentes necesidades del sistema.

Arquitectura distribuida

Idea principal

Una arquitectura distribuida organiza el sistema en **componentes separados** que se ejecutan en distintos procesos, servidores o servicios, y se comunican mediante red.

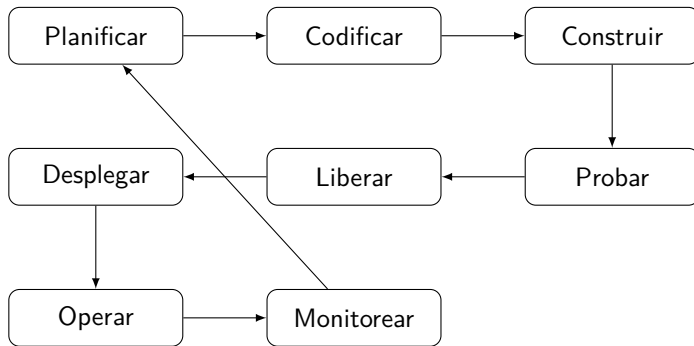


Definición

DevOps es un enfoque cultural y técnico que integra **desarrollo** y **operaciones** para entregar software de forma más rápida, confiable y automatizada.

Prácticas principales

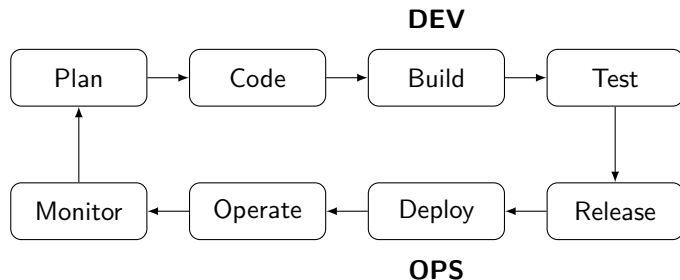
- Control de versiones con Git.
- Integración continua.
- Entrega o despliegue continuo.
- Automatización de pruebas.
- Contenedores e infraestructura como código.
- Monitoreo, métricas y alertas.



Idea clave

DevOps permite que los cambios arquitectónicos y funcionales se entreguen con menor riesgo mediante automatización, validación y observabilidad.

Ciclo continuo de DevOps



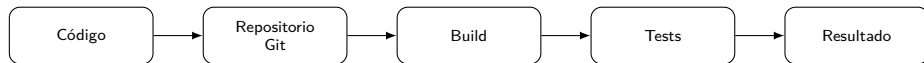
Retroalimentación continua

La información obtenida durante la operación y el monitoreo vuelve al equipo para planificar nuevas mejoras, corregir errores y generar nuevas versiones.

CI: Continuous Integration

Integración Continua

Continuous Integration (CI) consiste en integrar frecuentemente los cambios realizados por los desarrolladores y validarlos automáticamente.



Un pipeline CI puede ejecutar automáticamente:

- compilación o construcción;
- pruebas unitarias y de integración;
- análisis estático de código;
- validaciones de calidad;
- generación de artefactos.

Objetivo

Detectar rápidamente si un cambio introduce errores o rompe la integración con el resto del sistema.

Tecnologías utilizadas en Integración Continua

CI — Continuous Integration

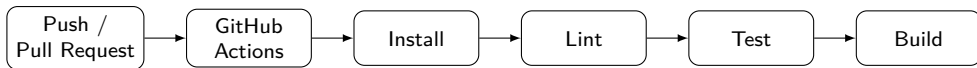
La integración continua combina diferentes herramientas para **construir, probar y validar automáticamente** cada cambio incorporado al repositorio.

Función	Tecnologías
Control de versiones	Git, GitHub, GitLab
Motor de CI	GitHub Actions, GitLab CI/CD, Jenkins, Azure Pipelines
Build / dependencias	npm, Maven, Gradle
Pruebas automatizadas	Jest, Vitest, JUnit, pytest
Calidad de código	ESLint, SonarQube
Empaquetado	npm, Docker

Idea clave

La plataforma de CI **orquesta el pipeline**; las demás herramientas realizan las tareas de construcción, pruebas y validación.

Ejemplo de CI con GitHub Actions



Ejemplo Angular / NestJS

- `npm ci`: instala las dependencias.
- `npm run lint`: verifica calidad del código.
- `npm test`: ejecuta pruebas automatizadas.
- `npm run build`: comprueba que la aplicación pueda construirse correctamente.

Resultado

Si alguna etapa falla, el pipeline informa el error y el cambio no debería integrarse hasta ser corregido.

Ejemplo: GitHub Actions para CI

El archivo se almacena en: `.github/workflows/ci.yml`

```
name: CI
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Obtener código
        uses: actions/checkout@v6
      - name: Configurar Node.js
        uses: actions/setup-node@v7
        with:
          node-version: '20'
      - name: Instalar dependencias
        run: npm ci
      - name: Ejecutar pruebas
```

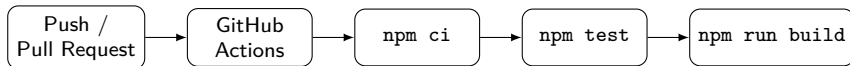
¿Cómo interpretar el workflow?

Elemento	Función
name	Nombre que identifica el workflow.
on	Define qué evento inicia automáticamente el pipeline.
push	Ejecuta CI cuando se envían cambios al repositorio.
pull_request	Ejecuta CI antes de integrar cambios mediante un Pull Request.
jobs	Define los trabajos que ejecutará el pipeline.
runs-on	Indica el entorno donde se ejecutará el job.
steps	Secuencia de acciones y comandos que componen el job.

Estructura

Workflow → **Evento** → **Job** → **Steps** → **Actions** / comandos

¿Qué ocurre cuando se realiza un Push?



Resultado

Si todas las etapas son exitosas, el cambio ha superado las validaciones definidas por CI.

Si una etapa falla

El pipeline se marca como fallido y el equipo puede corregir el problema antes de integrar el cambio.

CD: Continuous Delivery y Continuous Deployment

CD puede tener dos significados relacionados

Ambos continúan el proceso después de CI, pero se diferencian en cómo el software llega a producción.

Continuous Delivery

Entrega Continua

- Build y pruebas automáticas.
- Se genera una versión desplegable.
- Puede desplegarse automáticamente a staging.
- El paso a producción puede requerir **aprobación manual**.

Continuous Deployment

Despliegue Continuo

- Build y pruebas automáticas.
- Se genera una versión desplegable.
- Si todas las validaciones son exitosas, el cambio llega **automáticamente a producción**.

Diferencia fundamental

Delivery: el software queda listo para producción.

Deployment: el software se despliega automáticamente a producción.

Arquitectura frontend

Angular utiliza una **arquitectura basada en componentes**: la interfaz se divide en componentes reutilizables, apoyados por servicios, enrutamiento e inyección de dependencias.

Alternativa	Uso principal
Angular + Ionic + Capacitor	Aplicaciones móviles híbridas con componentes UI móviles.
Angular + Capacitor	Aplicación Angular web empaquetada para Android/iOS, sin usar Ionic.
Angular + NativeScript	Aplicaciones móviles con interfaz nativa usando Angular.
Angular PWA	Aplicación web instalable desde navegador, con soporte offline limitado.

Idea clave

Ionic no es la única opción para móviles con Angular: la elección depende de si se requiere UI híbrida, acceso nativo, reutilización web o experiencia nativa.

Arquitectura frontend con Angular

Idea principal

Angular organiza la aplicación mediante una **arquitectura basada en componentes**, donde la interfaz, la navegación y el acceso a datos se separan en responsabilidades.

